ParkM Phases 4 & 5 Detailed Plan

ParkM Zoho Desk Automation — Phases 4 & 5 Detailed Plan

TL;DR (one-page version)

- **Phases 1–3 are live in production** (Sadie pilot). The CSR wizard, AI tagging, and CSR-driven refund automation are working today.
- **Phase 4** = expand the "single pane of glass" beyond refunds to **every high-volume ticket type** (vehicle updates, account/payment, lockout/override, tow disputes). Today the wizard fully eliminates the parkm.app round-trip *only for refunds*. Phase 4 brings that same experience to the other ~80% of CSR ticket volume.
- **Phase 5** = **progressive automation toward Katie's stated end state: 100% bot-driven refund processing, no CSR involvement.** Structured as four explicit "crawl-walk-run" stages so risk stays contained and we can stop at any stage if results aren't compelling.
- **This work runs in parallel with Patrick's sales process and the launch-coordinator initiative** — different surface areas, no resource conflict, no shared critical path. ParkM doesn't have to choose one or the other.
- **Recommendation:** approve Phase 4 to start now (low-risk, high-leverage CSR productivity gains), and approve Phase 5 Stage 1 ("simple cancellations") so we can begin proving the automation pattern on the lowest-risk ticket type while Phase 4 ships in parallel. Phase 5 Stages 2–4 stay gated on Stage 1 results.

What's already in production (so we're scoping the *remaining* work)

Before scoping Phase 4 and 5, it's worth being explicit about what we already shipped in Phases 1–3, because some of what was originally proposed for Phase 4 is now done:

Capability	Status
AI classification + multi-tag (51 tags) on every inbound ticket	✅ Live
CSR wizard widget embedded in Zoho Desk ticket view	✅ Live
Wizard content for all 51 tags (from Sadie's "How to do everything ParkM" doc)	✅ Live
ParkM.app data embedded in wizard for **refund** tickets (customer lookup, permits, transactions, eligibility calc)	✅ Live
One-click permit cancellation (CSR confirms, system calls parkm.app `CancelPermit`)	✅ Live
Auto-generated refund forward email to `accounting@parkm.com` with all required details	✅ Live
Inactive-permit list with last charge date and refund-window status	✅ Live
41 response templates wired into wizard quick-reply	✅ Live
Zoho status writeback on action (e.g. "Waiting on Accounting")	✅ Live

What this means: the **plumbing** is there (parkm.app client, Zoho writeback, wizard engine, template engine). Phase 4 and 5 are about extending that plumbing to more ticket types (Phase 4) and removing the CSR from the loop entirely on the ones we trust (Phase 5).

PHASE 4 — Unified Agent Desktop (extend beyond refunds)

Goal

Eliminate the Zoho ↔ parkm.app context switch for **every** high-volume CSR workflow, not just refunds. Today a CSR handling a vehicle update or account question still has to flip to parkm.app to look things up. Phase 4 closes that gap.

Why this matters

- Refunds are ~20% of CSR ticket volume. The other 80% (vehicle updates, account questions, payment issues, lockouts, tows, etc.) still require the CSR to alt-tab to parkm.app for context.
- Sadie has already validated the wizard pattern works. Extending it to other tag types is the highest-leverage "more value, same architecture" investment we can make.

- Several of these flows (vehicle updates, account lookups) are also the **prerequisite data lookups for Phase 5 automation**. So Phase 4 work is a stepping stone — its outputs are reusable.

Scope — concrete deliverables

4.1 — Embed parkm.app context for the next tier of tag types - Vehicle Update tickets: pull customer + permits + active vehicle, render in wizard, one-click "Update Vehicle" via parkm.app API - Account / Login Issue tickets: customer lookup, account status, permit count, "Send Password Reset" one-click action - Payment Issue tickets: full transaction history (the same `Permits/GetAllPaymentsForPermit` we already use for refund eligibility), failed-payment detection, "Resend Receipt" action - Lockout / Override / Locked Permit tickets: permit status + override history, one-click "Issue Override" if API supports

4.2 — Property-side context (new data layer) - Many tickets are property-level rather than customer-level (sales-rep escalations, property-manager complaints, enforcement questions). Today the wizard has zero property context. - Add property lookup endpoint usage, render: property name, PM contact, total active permits, recent enforcement events - Enables the wizard to actually help on Sales-Rep and Property-tag tickets (currently 26 of our 51 tags are property/sales-rep — and the wizard for those is text-only)

4.3 — Bidirectional Zoho ↔ parkm.app sync - Today: actions write to parkm.app and Zoho separately. Phase 4 wraps these in a single transactional layer with rollback on partial failure. - Logging surface for Sadie to audit "what did the bot do on this ticket" — needed before Phase 5 can ride on top.

4.4 — Inline "Why am I seeing this?" reasoning panel - Show the AI's reasoning + extracted entities + which template was suggested and why - Required for CSR trust before we hand any decisions to the bot in Phase 5 - Doubles as a debugging tool for Sadie when classification is off

Phase 4 parallelism

Can run **fully in parallel** with Patrick's sales process work — different repo, different surface area, zero shared dependencies.

Phase 4 success metrics

- CSR can resolve **≥80% of all ticket types** without leaving Zoho Desk (today: ~25%, mostly refunds)
- Average ticket handle time: 30–40% reduction across non-refund tickets
- Zero increase in incorrect actions (audit log shows actions match CSR intent)

PHASE 5 — Progressive Refund Automation (toward 100% bot-driven)

Goal

Move refund processing from "CSR-driven with bot assistance" (today) to **"bot-driven with optional CSR review"** (Stage 4). This matches the end-state Katie articulated on the May 7 call.

Why this is structured as four stages

A single jump from "CSR clicks every step" to "no CSR involvement" is too aggressive — it's the same calculated rollout pattern Chad approved for Phases 1–3. We crawl, then walk, then run. Each stage is independently valuable. ParkM can stop at any stage and we still leave them better off than they are today. **No commitment to later stages until earlier stages prove out.**

Stage 5.1 — Auto-handle "simple cancellation, no refund" tickets

The case: Customer says some variant of "please cancel my permit, I moved out." No refund mentioned. Permit not yet cancelled in parkm.app. License plate or email lets us identify them unambiguously.

Bot does: 1. Detect intent (already done by classifier) 2. Look up customer in parkm.app (already done in wizard) 3. Confirm only one active permit + customer past their move-out date + no refund language detected (boolean checks) 4. If all green → call `Permits/CancelPermit?sendNotice=true`, send confirmation email using existing `cancellation_confirmed.html` template, close ticket as "Resolved by Bot" 5. If any red → leave for Sadie's queue, no harm done

Why start here: - Lowest-risk ticket type — no money moving, no legal exposure, parkm.app sends its own cancellation notice - High volume — Katie/Sadie data shows simple cancellations are a meaningful chunk of weekly tickets - Reversible — if a cancellation is wrong, CSR can reactivate the permit (we already have the `reactivate_all_permits.py` pattern) - Proves the automation pattern in production with maximum safety

Stop condition for Stage 5.2: Stage 5.1 must run for ≥2 weeks in production with **zero incorrect cancellations** before Stage 5.2 starts.

Stage 5.2 — Auto-handle "missing info" requests

The case: Customer requests a refund or cancellation but didn't include their license plate, or didn't include a bank-statement screenshot, or didn't include a move-out date.

Bot does: 1. Detect missing-info ticket via classifier 2. Pick the right template (`missing_license_plate.html`, `missing_bank_statement.html`, `missing_move_out_date.html` — already exist) 3. Send the reply automatically, set ticket to "Waiting on Customer" 4. When customer replies, ticket re-enters classification with the new info

Why second: - Same “no money moving” safety profile as Stage 5.1 - Templates are already written and Sadie-approved - No parkm.app writes — pure email automation - Eliminates a chunk of CSR work that’s pure copy-paste today

Stop condition for Stage 5.3: Stage 5.2 must show **≥95% customer satisfaction parity** with CSR-sent messages (measured via reply rates / negative-language detection in responses).

Stage 5.3 — Auto-submit refund-eligible tickets to accounting

The case: Customer requests a refund. Bot validates: single permit, within 30-day window, charge confirmed in parkm.app, no dispute language, license plate matches.

Bot does: 1. Run the full refund eligibility check that the wizard already runs today (zero new logic — just removing the CSR’s “Confirm” click) 2. Cancel the permit in parkm.app 3. Forward the refund request to `accounting@parkm.com` using the existing `refund_forward_accounting.html` template 4. Set ticket to “Waiting on Accounting” 5. CSR is notified (not asked to act) so Sadie can spot-audit

Why third: - This is just removing one click from a flow the wizard already does correctly today - Accounting still does the actual money movement (Reverse Charge in parkm.app) — bot doesn’t touch financials - All the safety checks that exist in the wizard today carry over unchanged - Sadie’s audit log (built in Phase 4.4) makes this easy to monitor

Stop condition for Stage 5.4: Stage 5.3 must run for ≥4 weeks with **zero incorrect submissions** AND accounting team must explicitly sign off that the bot-submitted forwards are higher-quality than CSR-submitted forwards.

Stage 5.4 — End-to-end refund automation including accounting

The case: Same as Stage 5.3, but the bot also performs the Reverse Charge in parkm.app (currently done manually by accounting) and sends the customer their refund confirmation.

Bot does: - Everything in Stage 5.3, plus: - Call parkm.app’s reverse-charge endpoint (assuming Stephen confirms one exists or builds one — this is the **single biggest dependency for Stage 5.4**) - Send `refund_approved.html` to the customer with the 5-business-day refund timeline - Close the ticket - Log everything for the accounting team’s daily reconciliation report

Why last: - This is the only stage where the bot moves money. It needs the most rigor. - Requires parkm.app to expose (or build) a reverse-charge endpoint — currently only available via the parkm.app UI - Requires accounting team buy-in on a daily reconciliation report instead of per-refund manual review - This is **Katie’s stated end state** — 100% bot-driven, zero CSR/accounting involvement on the happy path

Stop conditions for permanent rollout: - ≥6 weeks of Stage 5.4 in production with **zero financial errors** - Accounting team explicitly comfortable with the daily reconciliation flow - Chad sign-off on the final cutover

Phase 5 commitment model

Stage-by-stage. ParkM only commits to the next stage when the prior stage has met its stop condition — no upfront commitment to the full Phase 5 scope.

Phase 5 success metrics

- Stage 5.1: ≥30% of CSR’s daily ticket volume now bot-handled
- Stage 5.2: Pure additive — frees ~5 min/ticket × volume
- Stage 5.3: ≥80% of refund-eligible tickets handled without CSR click
- Stage 5.4: ≥80% of all refund tickets handled without **any** human until accounting reconciliation, fulfilling Katie’s stated end state

---

Recommended decisions for ParkM

I’m asking for **two decisions**, not one:

Decision 1 — Approve Phase 4 to start immediately. Low risk (extends a pattern already proven in production), high CSR-productivity payoff, and unblocks Phase 5.

Decision 2 — Approve Phase 5 Stage 1 (“simple cancellations”) to start in parallel with Phase 4. Smallest, safest piece of automation. Proves the bot can act autonomously on the lowest-risk ticket type. **All later Phase 5 stages stay gated on Stage 1 results — you’re not committing to anything beyond this with this decision.**

This structure lets ParkM commit to a small, well-scoped piece of autonomous-bot work *now* without committing to the full end-state until the data supports it — which is exactly the calculated rollout style Chad has already endorsed.